

Article

The Choice of Training Data and the Generalizability of Machine Learning Models for Network Intrusion Detection Systems

Marcin Iwanowski * , Dominik Olszewski , Waldemar Graniszewski , Jacek Krupski  and Franciszek Pelc

Institute of Control and Industrial Electronics, Warsaw University of Technology, ul.Koszykowa 75, 00-662 Warszawa, Poland; dominik.olszewski@pw.edu.pl (D.O.); waldemar.graniszewski@pw.edu.pl (W.G.); jacek.krupski.dokt@pw.edu.pl (J.K.); franek.pelc@gmail.com (F.P.)

* Correspondence: marcin.iwanowski@pw.edu.pl

Abstract

Network Intrusion Detection Systems (NIDS) driven by Machine Learning (ML) algorithms are usually trained using publicly available datasets consisting of labeled traffic samples, where labels refer to traffic classes, usually one benign and multiple harmful. This paper studies the generalizability of models trained on such datasets. This issue is crucial given the application of such a model to actual internet traffic because high-performance measures obtained on datasets do not necessarily imply similar efficiency on the real traffic. We propose a procedure consisting of cross-validation using various sets sharing some standard traffic classes combined with the t-SNE visualization. We apply it to investigate four well-known and widely used datasets: UNSW-NB15, CIC-CSE-IDS2018, BoT-IoT, and ToN-IoT. Our investigation reveals that the high accuracy of a model obtained on one set used for training is reproducible on others only to a limited extent. Moreover, benign traffic classes' generalizability differs from harmful traffic. Given its application in the actual network environment, it implies that one needs to select the data used to train the ML model carefully to determine to what extent the classes present in the dataset used for training are similar to those in the real target traffic environment. On the other hand, merging datasets may result in more exhaustive data collection, consisting of a more diverse spectrum of training samples.

Keywords: intrusion detection system; internet traffic classification; machine learning



Academic Editors: Emil Pricop and Khalil El-Khatib

Received: 7 July 2025

Revised: 24 July 2025

Accepted: 25 July 2025

Published: 30 July 2025

Citation: Iwanowski, M.; Olszewski, D.; Graniszewski, W.; Krupski, J.; Pelc, F. The Choice of Training Data and the Generalizability of Machine Learning Models for Network Intrusion Detection Systems. *Appl. Sci.* **2025**, *15*, 8466. <https://doi.org/10.3390/app15158466>

Copyright: © 2025 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (<https://creativecommons.org/licenses/by/4.0/>).

1. Introduction

Network Intrusion Detection Systems (NIDS) play a vital role in maintaining the appropriate level of cybersecurity for contemporary computer systems. Their primary objective is identifying unauthorized access or irregularities in network traffic and system operations. The emergence of new forms of Internet communication, such as the Internet of Things (IoT) that connects various devices and Operational Technology (OT) that oversees and controls physical devices, processes, and events within industrial settings, has not only amplified the volume of network traffic but also diversified it, thereby introducing new paths for cyber threats [1]. By detecting suspicious behaviors, NIDS contributes significantly to preventing security breaches.

Our investigation focuses on intrusion detection systems based on the online processing of traffic features that use pre-trained Machine Learning (ML) models. In the vast majority of cases, such systems are trained on the collected labeled traffic data samples before their proper use. As in most ML models, the quality of the training data is one of

the most significant factors influencing the overall final efficiency and accuracy of the machine-learning-based NIDS. Typically, the acceptable measures obtained on the test data, usually of the exact origin as the training one, make the model ready to deploy. Finally, considering the correct (in terms of performance measures) work on the test data, one expects the model's good performance on actual network traffic. It is the principal factor influencing the process of transition of the model from the "closed world" of the training environment into the real world of actual network traffic [2]. Such a transfer-learning approach is common in all fields of machine learning, not excluding the NIDS [3]. In our paper, we show that this process does not necessarily run smoothly in the case of the ML model trained on the available internet traffic datasets, which is primarily due to a wide variety of data transmitted over the network from one side and relatively limited scope of data gathered on the other.

In our research, we focus on the dataset bias that influences the ability of the NIDS to generalize its expertise. This property is crucial in all ML models as it allows for detecting traffic classes on data of the same categories from sources other than the training data. We investigate the generalizability of four popular Internet traffic data sets: UNSW-NB15 [4], CSE-CIC-IDS2018 [5], BoT-IoT [6] and ToN-IoT [7] in their unified ('v2') versions sharing the same collection of features [8] (In the remainder of the paper these four sets are referred to by the following simplified abbreviations as NB15, IDS18, BoT and ToN).

The first two datasets collect traffic that circulates in typical enterprise networks, while the remaining two contain samples of the industrial IoT network traffic. Although registered in controlled environments, they all reflect real-world scenarios and challenges, making them valuable resources for academic research and practical cybersecurity applications. They contain specific data of different origins consisting of various traffic classes, including benign traffic and multiple harmful categories. The categories differ from one set to another, but some classes are present in more than one dataset (co-occurring classes). We focus on them, observing how these classes are detected within samples belonging to one set while training the model on another.

In our experiments, we use the extremely randomized trees (extra trees) classifier [9] to investigate the influence of train-test set combinations on traffic classification efficiency. This investigation is based on the cross-validation of models using subsets of original datasets consisting of one million samples and a reduced number of features each. Results reveal that the high accuracy of detecting certain classes exhibited on the test set (originated from the same data collection as the one from which the training set comes) is, in some cases, followed by low accuracy obtained on the data from other collections.

To support and complete the above analysis, we apply a data visualization of classes co-occurring in different datasets. The t-distributed Stochastic Neighbor Embedding (t-SNE) dimensionality reduction method [10] is applied to reduce the original multidimensional feature space into easy-to-visualize two dimensions. Obtained visualizations explain the low accuracy of specific training and test dataset combinations caused by intra-dataset diversity within co-occurring classes.

The paper is organized as follows. Section 2 contains a short survey of related works. Section 3 focuses on the traffic data used in the current study. Section 4 describes the methodology. Section 5 presents the results obtained and a discussion on them. Finally, Section 6 concludes the paper.

2. Related Works

The research on this topic is significant, as it is caused by the rapid development of ML methods and applications on one side and the instantly growing network traffic with the accompanying increase in cyber threats on the other [1,11–14]. Also, relatively new networks like the Internet of Things (IoT), connecting consumer devices, and the Industrial Internet of Things (IIoT), which are gaining popularity in the industrial domain, have become vulnerable to attacks. As a result of all the above factors, many methods and approaches have been developed till now [15–19].

The paper [20] explores the relationship between NIDS models and datasets, identifying biases introduced by training on non-comprehensive datasets. The authors conclude that model overfitting and dataset limitations significantly reduce the generalizability of NIDS models, calling for improvements in both model robustness and dataset quality. The paper [21] evaluates the inter-dataset generalizability of autoencoders for NIDS, focusing on their ability to perform well across different datasets, highlighting the challenge of dataset-specific biases in model performance. Paper [22] performs a cross-dataset evaluation of ML models for intrusion detection. The study reveals that many ML models exhibit poor cross-dataset performance, stressing the need for better generalization strategies. Dataset diversity and data quality are highlighted as critical factors influencing model performance. Paper [23] evaluates standard feature sets, improving the generalizability and explainability of ML-based NIDS. The findings suggest that carefully selected universal features can help models generalize better across datasets.

Studies [24,25] focus on the explainable cross-domain evaluation of ML-based NIDS, emphasizing the challenges of transferring models across different network environments. Papers recommends cross-domain validation to assess real-world effectiveness. Article [26] analyzes the inter-dataset generalization strength of supervised ML models for intrusion detection. The study by [27] experiments on generalizability with four datasets (pairwise) with visualization. It examines the cross-dataset generalization of ML models for NIDS, finding that models trained on one dataset often struggle when tested on others. Paper [28] discusses dataset biases in learning systems for intrusion detection, highlighting how dataset-specific characteristics can distort model performance. In [29], authors investigate the issue of overfitting various ML models that cause unfounded high accuracy on training data in terms of the data or pattern leakage. In contrast to the relatively low number of papers discussing the issue of generalizability, the number of papers focusing on developing the most effective ML-based IDS models is growing rapidly [30].

There are many datasets used in ML-based NIDS. Papers [31] presented their comprehensive surveys. In [32–34], the authors discuss various aspects of collecting and processing data, given the need for efficient machine learning datasets. Considering only four datasets used in our paper, many recent papers have described newly proposed models.

The NB15 dataset has been used in many research projects. In Industry 4.0 applications, the dataset in question was utilized to train models that intelligently detect intruders [35] and to find anomalies [36]. The problem of protection of the heterogeneous Internet of Things was discussed in [37]. Similarly, in [38], the dataset tested a system to protect the Internet of Things from rare or completely new attacks. It was also applied to validate the high-speed railway sensor defense [39]. In [40], a dataset was used to create a system to protect the Internet of Things from DoS or DDoS-type attacks. The dataset was also utilized to test an anomaly discovery model for IoT [41] and used to prove the properties of novel deep-learning models [42,43]. The study [44] presents an NIDS model based on a soft voting method for detecting intrusion. The method based on combination of CNN and GRU was proposed in [45].

The IDS18 dataset was used to analyze the problem of intrusion detection in IoT [46] and Industrial IoT (IIoT) [47]. Paper [48] presents a method that combines signature-based and AI-powered deep analysis, called PAID, controllable by a flow sensing strategy for accelerating intrusion detection in large-scale networks. In [49], a Deep Feature Fusion Convolutional Neural Network (DFFCNN) model combined with data preprocessing to execute deep DDoS attack detection is proposed. Paper [50] focuses in bio-inspired optimization techniques for NIDS. For survey of deep learning frameworks used in network intrusion detection based on this set, see [51].

The ToN dataset was used to find, e.g., anomalies [41] and intruders in IoT [52], as well as to validate (along with, among others, NB15) novel deep-learning models [53]. In another study, where blockchain was utilized to prevent poisoning-type attacks, this dataset was used to test how the proposed system performs in recognizing patterns of network attacks [54]. ToN can be found in a study on privacy in IIoT [55] and in a research paper on the standardization of features and types of attacks in IoT [56]. In addition, the dataset was used in a study on security in medical systems: medical IoT [44] and industrial healthcare system [57].

The BoT dataset was applied in a study focused on the detection of less popular and new previously unknown aspects of the IoT: attacks [38], intrusion detection system [41], and detection of botnets IoT [58].

In many papers, multiple datasets are used to validate ML models for NIDS. In [59], a hybrid machine learning model for intelligent cyber threat identification in smart city environments is proposed and tested on BoT and ToN datasets. In [60], both sets are used to test the lightweight model for DDoS attack detection. In [61], the NB15 and ToN datasets are used to verify the method based on convolutional backbone neural networks. The paper [62] proposes the evolutionary approach for enhanced attack detection and classification that is verified using, among others, NB15, BoT, and IDS18 datasets. In [63], the concept of spatial-temporal fusion gating a multilayer perceptron for network intrusion detection is proposed and tested on NB15, BoT, and ToN. Paper [64] employs NB15, IDS18, and three other datasets to validate two deep generative models designed to enhance learning latent features for detecting network anomalies.

3. Datasets and Features

There is a high variety of available datasets gathering network traffic. In our study, we have chosen four of them. The choice was motivated by two principal factors. First, they should represent various types of traffic, including classic traffic, typical for business environments, and industrial IoT traffic. Secondly, all sets should have the same data format. As we focus on network features (not the raw traffic), they all should contain the same, unified set of traffic features. Our investigation uses four public datasets designed to train IDS systems, fulfilling the above requirements. Two of them, NB15 and IDS18, contain traffic circulating in typical enterprise networks. The remaining two, BoT and ToN, consist of traffic registered in the Internet of Things networks and address the growing need for robust cybersecurity solutions in IoT and IIoT environments, which are more complex and varied than traditional IT networks [65].

All four datasets have been captured in controlled network environments specially designed to generate various types of traffic. They contain traffic samples belonging to various classes: single benign and multiple harmful. The composition of the latter depends on the datasets. Sets are also imbalanced—the number of samples in each class differs. The list of available classes and their share in the total number of samples of particular datasets is shown in Table 1.

Table 1. Percentages of the total number of samples belonging to particular classes in each dataset (columns sum up to approx. 100% as tiny amounts are neglected, co-occurring classes in bold).

Class	BoT	IDS18	NB15	ToN
Analysis	0	0	0.1	0
Backdoor	0	0	0.09	0.1
Benign	0.36	88.05	96.02	36.01
Bot	0	0.76	0	0
Brute Force	0	0.66	0	0
DDoS	48.54	7.36	0	11.96
DoS	44.15	2.56	0.24	4.21
Exploits	0	0	1.32	0
Fuzzers	0	0	0.93	0
Generic	0	0	0.69	0
Infiltration	0	0.62	0	0
Reconnaissance	6.94	0	0.53	0
Shellcode	0	0	0.06	0
Theft	<0.01	0	0	0
Worms	0	0	<0.01	0
Injection	0	<0.01	0	4.04
Mitm	0	0	0	0.05
Password	0	0	0	6.81
Ransomware	0	0	0	0.02
Scanning	0	0	0	22.32
XSS	0	0	0	14.49

3.1. Datasets

The NB15 dataset [4] comprises 100 GB of raw network traffic, organized in over two million registered traffic records. It contains nine types of attacks and the *Benign* traffic class that remarkably dominates over the harmful traffic classes. Apart from the original raw traffic data, the dataset contains 49 features for each traffic sample. The dataset has been used in many research projects related to both enterprise and industrial traffic.

The IDS18 dataset [5] includes traffic generated by multiple devices, such as Windows, Linux, and MacOS-based machines, as well as various servers and client systems. The attacks are categorized into six different types. The dataset also includes regular, benign traffic representing typical user behavior in an enterprise network. The dataset initially included 80 features extracted from the captured traffic.

The ToN dataset [7] is a collection of data for the evaluation of cybersecurity applications in the context of the Internet of Things (IoT) and Industrial IoT (IIoT). It is designed to assess the performance of various AI-based cybersecurity applications, including intrusion detection systems, malware detection, and threat intelligence. The dataset includes diverse data sources, such as network traffic data (captures the communication between devices in the IoT/IIoT network), telemetry data (logs from various IoT devices, capturing their operational states and activities), operating system logs (collected from systems involved in the IoT/IIoT environment, such as edge and cloud systems), and application logs (contains

data generated by applications running on IoT/IoT devices). The dataset includes nine cyber-attacks that are relevant to IoT/IoT environments.

The BoT dataset [6], similarly to ToN, focusing on the IoT traffic, contains over 72 million records stored initially in pcap files. The testbed environment included typical IoT devices like smart lights, motion sensors, and other smart appliances. The dataset includes a range of cyber attacks relevant to IoT environments that are categorized into four main types and benign traffic. The dataset originally included 42 features extracted from the network traffic representing various aspects of network communication.

3.2. Unified Set with Meaningful Features

The above datasets vary in the collections of features they consist of. Their comparison requires that they all share the same collection of features. To address this issue, the new version of these sets was proposed in [8]. In the new versions, each dataset contains 43 unified features (see Table 2 for their list). The suffix ‘v2’ was added to set names to differentiate new sets from old ones.

Following [8], in our study, the original data was preprocessed to remove the features that could unjustifiably influence the classification results—the misleading features and redundant—highly correlated—features. In particular, all features that are flow identifiers, like source and destination IP addresses and port numbers (4 features), time-related features (5), and TTL-based features (3), were removed. The collection of features after this step consists of 31 items. Next, the correlation analysis was performed to minimize the number of features by removing the redundant ones. Computing the correlation matrix of all feature pairs allowed for finding the strongest correlations. The pairs of very strong correlation, i.e., with correlation coefficient ≥ 0.8 , contained candidates for possible removal. The one with the lowest correlation with the other features was finally removed among each of the two candidates. All the removed features, along with appropriate correlations, are listed in italics in Table 3. As there were seven such features, the ultimate number of features is 24. The complete list of traffic features are shown in Table 2; removed ones are printed in italics.

Table 2. List of features in the ‘v2’ versions of datasets. Items that are not used in experiments printed in italics. Letter after the ordinal number indicate the reason for removal: m—misleading, r—redundant feature.

	Feature name	Description
1 (m)	<i>IPV4_SRC_ADDR</i>	<i>IPv4 source address</i>
2 (m)	<i>L4_SRC_PORT</i>	<i>IPv4 source port number</i>
3 (m)	<i>IPV4_DST_ADDR</i>	<i>IPv4 destination address</i>
4 (m)	<i>L4_DST_PORT</i>	<i>IPv4 destination port number</i>
5	PROTOCOL	IP protocol identifier byte
6	L7_PROTO	Application protocol as a number
7	IN_BYTES	Incoming number of bytes
8	IN_PKTS	Incoming number of packets
9	OUT_BYTES	Outgoing number of bytes
10 (r)	<i>OUT_PKTS</i>	<i>Outgoing number of packets</i>
11	TCP_FLAGS	Cumulative of all TCP flags

Table 2. Cont.

	Feature name	Description
12 (r)	CLIENT_TCP_FLAGS	Cumulative of all client TCP flags
13	SERVER_TCP_FLAGS	Cumulative of all server TCP flags
14 (m)	FLOW_DURATION	Flow duration in milliseconds
15 (m)	DURATION_IN	Incoming stream duration in millisec.
16 (m)	DURATION_OUT	Outgoing stream duration in millisec.
17 (m)	MIN_TTL	Minimal flow TTL
18 (m)	MAX_TTL	Maximal flow TTL
19 (m)	LONGEST_FLOW_PKT	Longest packet (bytes) of the flow
20	SHORTEST_FLOW_PKT	Shortest packet (bytes) of the flow
21	MIN_IP_PKT_LEN	Smallest flow IP packet len observed
22	MAX_IP_PKT_LEN	Largest flow IP packet len observed
23	SRC_TO_DST_SECOND_BYTES	Src to dst bytes/sec
24	DST_TO_SRC_SECOND_BYTES	Dst to src bytes/sec
25	RETRANSMITTED_IN_BYTES	No. of retr. TCP flow bytes (src-dst)
26 (r)	RETRANSMITTED_IN_PKTS	No. of retr. TCP flow packets (src-dst)
27	RETRANSMITTED_OUT_BYTES	No. of retr. TCP flow bytes (dst-src)
28 (r)	RETRANSMITTED_OUT_PKTS	No. of retr. TCP flow packets (dst-src)
29 (m)	SRC_TO_DST_AVG_THROUGHPUT	Src to dst average throughput (bps)
30 (m)	DST_TO_SRC_AVG_THROUGHPUT	Dst to src average throughput (bps)
31	NUM_PKTS_UP_TO_128_BYTES	Packets of IP size ≤ 128
32	NUM_PKTS_128_TO_256_BYTES	Packets of IP size > 128 & ≤ 256
33	NUM_PKTS_256_TO_512_BYTES	Packets of IP size > 256 & ≤ 512
34	NUM_PKTS_512_TO_1024_BYTES	Packets of IP size > 512 & ≤ 1024
35 (r)	NUM_PKTS_1024_TO_1514_BYTES	Packets of IP size > 1024 & ≤ 1514
36	TCP_WIN_MAX_IN	Max TCP Window (src-dst)
37	TCP_WIN_MAX_OUT	Max TCP Window (dst-src)
38 (r)	ICMP_TYPE	ICMP Type $\cdot 256 + \text{ICMP code}$
39	ICMP_IPV4_TYPE	ICMP Type.
40	DNS_QUERY_ID	DNS query transaction Id.
41	DNS_QUERY_TYPE	DNS query type (e.g., 1 = A, 2 = NS etc.)
42 (m)	DNS_TTL_ANSWER	TTL of the first A record (if any)
43	FTP_COMMAND_RET_CODE	FTP client command return code

Table 3. List of feature pairs with the strongest correlation > 0.8 .

Feature 1 (to Remove)	Feature 2 (Highly Corr.)	Corr. Coeff.
LONGEST_FLOW_PKT	MAX_IP_PKT_LEN	1.0
ICMP_TYPE	ICMP_IPV4_TYPE	0.9999
CLIENT_TCP_FLAGS	TCP_FLAGS	0.9962
NUM_PKTS_1024_TO_1514_BYTES	OUT_BYTES	0.9894
RETRANSMITTED_OUT_PKTS	RETRANSMITTED_OUT_BYTES	0.9893
OUT_PKTS	OUT_BYTES	0.8949
NUM_PKTS_1024_TO_1514_BYTES	OUT_PKTS	0.8863
RETRANSMITTED_IN_PKTS	RETRANSMITTED_IN_BYTES	0.8567

4. Methodology

In our research, we employ classification with inter-dataset cross-validation in two variants. The first one is based on the multi-class classification, considering all classes. It may, however, produce biased results due to class imbalance and a relatively low number of co-occurring classes, i.e., classes present in both sets used in a particular step of cross-validation. To overcome this problem, we perform a series of binary classifications, one per each co-occurring class. To better understand the outcomes of cross-validated classification, we use the t-SNE visualization that reduces the number of feature space dimensions into easy-to-visualize two. The scatter plot showing the distribution of points following their origin set allows for investigating the relations between data. The distribution of patterns visible on t-SNE charts explains the results obtained in the classification track and confirms the conclusions drawn.

4.1. Classifier(s) and Cross-Validation

One may find various classification models in reported research on NIDS [66]. However, when investigating the cross-dataset generalization properties described in some papers, one may observe a relatively high correlation between results obtained by different classifiers [27]. Although the results of individual tested classifiers differ, the relations between the results for individual sets usually remain constant. Our research aims not to find the best classifier but to investigate the generalizability of models trained on various datasets. That is why, in our paper, we omit the—typical in many papers in this domain—comparison of various classifiers, focusing on just a single one, carefully selected. Consequently, although several classifiers were initially investigated, a single one was chosen to perform the ultimate experiments. We pre-selected classifiers, restricting their broad spectrum to tree-based models. This choice was motivated by these models' high explainability and effectiveness. We have chosen three decision-tree-based classifiers for our experiments: decision trees [67], random forests [68], and randomized decision trees (extra-trees) classifier [9].

A decision tree [67] represents the classification process by a tree-based model. Each internal node of the tree corresponds to a test on a feature, each branch to a decision rule, and each leaf node represents a classification result. One of the main advantages of this approach is the ease of interpretation and understanding, as it is close to human decision-making processes. A random forest [68] is a classic ensemble learning method that operates by constructing a multitude of decision trees at training time and outputting the class that is computed, usually as a modal value, based on classification results of the individual trees. Random forest combine the predictions of several base decision-tree estimators built with a given learning algorithm to improve generalizability and robustness over a single classifier. Extremely randomized (extra-) tree [9] is a decision tree that introduces randomness as part of the construction process. Unlike traditional decision trees, where the best split at each node is determined based solely on deterministic criteria (like information gain or Gini impurity), a randomized decision tree incorporates random choices in splitting nodes.

We used the scikit-learn (1.5.2) Python (3.10.9) package in our experiments, where all three classifiers are available. Our goal was not to maximize the classification measures but to compare their performance for different train-test set combinations. The default and fine-tuned versions were used in the initial phase of the research to confirm the validity of such an approach. However, comparing results for default and fine-tuned versions revealed that although the latter generally provides higher values of quality measures, the relation between results obtained for various sets remains unchanged.

In the final phase of the classifier selection process, the above classifiers were trained and tested on various configurations of train-test sets. Again, the results revealed a high

correlation (between 0.75 and 0.95). Finally, the ExtraTree classifier was chosen as the one with the highest value for quality measures. This choice confirmed the choice of reference classifier applied in validation experiments in [8].

Our experiments investigate how classification models trained on one dataset perform when the other is used as a test set. As each set (BoT, ToN, NB15, and IDS18) was captured in different network testbeds, they allowed the simulation of applying the model trained on a given set used in a real environment.

Despite the various quantities of samples initially included in each of the four sets, equally sized subsets are extracted from each to perform the experiments. Thanks to that, we avoid the differences in results that may be caused by the inequality of training (and test) datasets' sizes. As the original sets are considerable (75,987,976 samples in total), subsets of a relatively large number of samples could have been extracted. Finally, two subsets (called subset A and subset B), consisting of one million samples each, were extracted from each original dataset. The choice was random while maintaining class proportions (see Table 1). Thus, one gets eight subsets of equal one million samples, two per every original dataset. Thanks to that, the results showed how the model behaves when tested on the subset originating from the same and different reference datasets. In addition, a separate experiment was conducted with the union of all four sets used to train the model. Based on this, the two-fold inter-dataset cross-validation was performed for each pair train-test set. It consists of two rounds. The first of two subsets extracted from the same original set was used as training, while the other was a test. In the second round, their roles interchange. Using two subsets from each original one, we got a more robust estimate of quality measures that are finally averaged. Figure 1 presents the processing scheme.

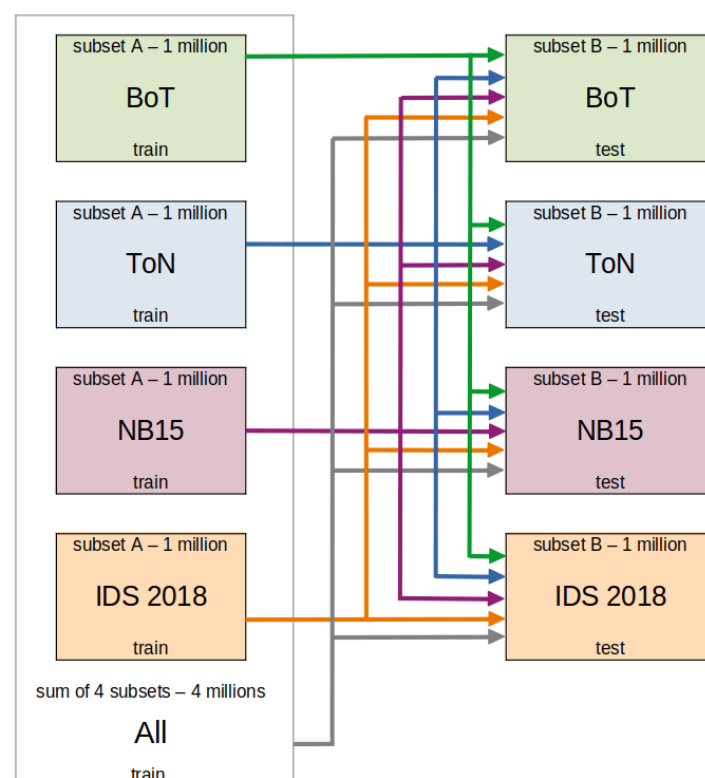


Figure 1. Cross-validation—first round, in the second round subset A and B are interchanged.

As seen in Table 1, the distribution of classes varies significantly across all datasets. The majority of classes are present in a single dataset. There are, however, some classes present in multiple sets. The above co-occurrent classes are *Benign*, *DoS*, *DDoS*, *Reconnais-*

sance, and Backdoor. Their presence allows for investigating the generalizability of a single class by computing appropriate quality measures for binary classification. In our experiments, we do not consider the fifth co-occurrent Injection class as its number of samples is tiny (Also, it is worth noting that some classes with different names contain traffic that may show significant similarities. An example of such a class is the Scanning class from the ToN dataset, similar to Reconnaissance from BoT and NB15. This issue is, however, not discussed in this paper).

4.2. Quality Measures

Our research investigates the behavior of classifiers working with all classes and single ones. The latter case refers to binary classification, where the selected class is confronted with the remainder. Consequently, the performance of classifiers should be measured using overall measures considering all classes (multiclass case) and using per-class ones (binary case). The usual per-class measures are accuracy, recall (sensitivity, detection rate), precision, and F1, defined as, respectively:

$$A_c = \frac{TP_c + TN_c}{TP_c + TN_c + FP_c + FN_c} = \frac{TP_c + TN_c}{n}, \quad (1)$$

$$R_c = \frac{TP_c}{TP_c + FN_c}, \quad P_c = \frac{TP_c}{TP_c + FP_c}, \quad F1_c = \frac{2 \cdot P_c \cdot R_c}{P_c + R_c}, \quad (2)$$

where lower index c stands for the class for which the measure is computed, TP , FP , TN , FN for numbers of true positive, false positive, true negative, and false negatives samples, respectively, and n stands for the total number of samples.

Apart from the above measures, there are other measures that focus on classification errors. Two of them, useful when measuring the performance of ML-based NIDS, are false negative rate (miss rate) and false positive rate (false alarm rate), which are defined respectively as:

$$FNR_c = \frac{FN_c}{FN_c + TP_c}, \quad FPR_c = \frac{FP_c}{FP_c + TN_c}. \quad (3)$$

An important question concerning the various measures is: Which is the best choice to properly validate the classifier for NIDS? To answer this question, one must first define some principles the classifier used in the NIDS must follow and apply them in the correct order. There are, in fact, two such principles. At first, the classifier must provide the highest possible level of protection, which means that as many harmful traffic samples as possible must be correctly classified, even if some benign ones are misclassified as harmful. Having met this requirement, the second principle should be fulfilled—one needs to take care of false alarms, i.e., the number of benign traffic samples misclassified as harmful should be low. The first principle directly implies that, in the case of classes of harmful traffic, the FN must be as low as possible. Only in the second place should the FP be taken into account. However, the order should be inverted in the *Benign* traffic class case. In this case, the number of FP should be as small as possible to fulfill the first requirement. It is because the sample incorrectly classified as *Benign* is, in fact, a harmful one that cannot be accepted. Only in the second place should FN be taken into account. Considering the above, one must differentiate measures to validate harmful and benign classes. In the first case, recall R_c and FNR_c are more suitable based on FN_c . In the second case, the benign traffic, one should use precision P_c and FPR_c based on FP_c .

Only the accuracy measure is equivalent to the binary case in the multi-class case. It is computed as the ratio of correct classifications to the total number of samples. In the case of other measures, the binary classifiers must be evaluated before computing the final

score regarding precision and recall. In our experiments, we applied weighted measures, defined as:

$$M = \sum_{c \in \text{Classes}} \frac{n_c}{n} M_c, \quad (4)$$

where M_c stands for measure of class c , which can either be precision (P_c) or recall (R_c), n_c for the number of samples in class c and n is the total number of samples.

4.3. Visualization

To further analyze the obtained results, focusing, in particular, on the descent of accuracy in some combinations of training and test sets, we apply a t-SNE dimensionality reduction approach [10] based on its predecessor, the SNE method [69]. It allows us to produce 2-dimensional visualizations, showing the distribution of samples of a given class originating from various datasets. This technique effectively reduces data complexity while preserving the data points' local structure. It converts the high-dimensional Euclidean distances between data points into conditional probabilities representing similarities. The algorithm then performs a low-dimensional embedding and finally places similar points closer together, locating dissimilar points further apart.

The t-SNE dimensionality reduction technique belongs to a group of neighborhood-preserving projections. It means that the local vicinities of the input and output data points are analyzed to preserve them. t-SNE can also be characterized as a probabilistic approach because it constructs probability distributions describing the local neighborhoods of the data points so that the distributions show the probability of a given data being in the neighborhood of the currently considered data point.

The t-SNE method improves the classical Stochastic Neighbor Embedding (SNE) approach. The original SNE algorithm uses the Gaussian probability distributions in both—input and output spaces. It preserves the neighborhood from the input data space in the output data space by minimizing the Kullback—Leibler divergence between the probability distributions. The extension introduced in t-SNE solves a significant difficulty associated with the standard SNE technique. Namely, SNE is vulnerable to the so-called “crowding problem” originating from a distortion in the low-dimensional manifold approximation in the output space. The distortion appears when a non-2-dimensional manifold is constructed due to embedding malfunction, and SNE cannot map the considerable dissimilarities correctly. Consequently, certain inaccuracies occur on the visualization display, typically gathering the points in the central part of the display. The “crowding problem” is discussed in detail in the work [10]. However, it has also been considered and analyzed in the paper [70].

In order to overcome the “crowding problem”, in [10], the Gaussian probability distribution was replaced by the Student's probabilistic t-distribution with one degree of freedom in the output data space. By doing so, they achieved an additional advantage besides the “crowding problem” limiting, accelerating the optimization process, i.e., the minimizations of the Kullback—Leibler divergences between the probability distributions in the respective local neighborhoods.

The ability of the t-SNE method to help observe data properties initially represented by samples in multidimensional space. A two-dimensional t-SNE plot of a dataset with several classes visualizes how samples relate to one another based on their high-dimensional features. Clusters on the plot represent groups of similar samples, meaning that points close together are more alike in the original feature space. When clusters are tightly packed and well-separated, it suggests that the corresponding classes are distinct and easily separable. Conversely, if clusters overlap or appear diffuse, this may indicate similarity between classes or ambiguity in the underlying features, potentially reflecting challenges

in classification. This is also the case of network traffic features, where it is successfully applied to investigate the properties of classes distributions in traffic feature space [71–75]. Our experiments use it to analyze the intra-class variability of samples from different datasets by reducing the 24-dimensional feature space into its two-dimensional equivalent.

5. Experiments

Experiments on generalizability consist of two phases. In the first one, the multi-class case is considered. The obtained results suggest limited but existing generalizability. It is further investigated to explore it more deeply, focusing on the *Benign* class, which is shown to be the main reason for such multi-class results. In the second phase of experiments, the co-occurring classes, i.e., classes present in more than one original set, are investigated to check how the generalizability looks in their case. The results reveal that the issue of generalizability is more complex than one may expect—it differs depending on the type of traffic class (benign/harmful).

5.1. Multi-Class Classification and Binary Benign-Class Case

In the first phase of experiments, we focused on testing the overall quality of cross-validated classifiers, measuring the multi-class measures for all combinations of training and test sets. In addition, the fifth case was introduced—the classifier was trained using a sum of all four training sets. In each case, the final measure is the mean value of two tests (2-fold cross-validation). Table 4 presents the results.

Table 4. Multiclass classification results

Train	Test	P	R	F1	A
BoT	BoT	0.985	0.985	0.985	0.985
BoT	ToN	0.075	0.038	0.050	0.038
BoT	NB15	0.931	0.243	0.380	0.243
BoT	IDS18	0.908	0.446	0.584	0.446
ToN	BoT	0.000	0.004	0.000	0.004
ToN	ToN	0.955	0.955	0.955	0.955
ToN	NB15	0.920	0.896	0.908	0.896
ToN	IDS18	0.769	0.788	0.778	0.788
NB15	BoT	0.036	0.003	0.000	0.003
NB15	ToN	0.133	0.350	0.187	0.350
NB15	NB15	0.990	0.989	0.989	0.989
NB15	IDS18	0.771	0.843	0.805	0.843
IDS18	BoT	0.082	0.003	0.000	0.003
IDS18	ToN	0.138	0.268	0.155	0.268
IDS18	NB15	0.922	0.958	0.940	0.958
IDS18	IDS18	0.992	0.992	0.992	0.992
All	BoT	0.976	0.947	0.954	0.947
All	ToN	0.953	0.953	0.952	0.953
All	NB15	0.990	0.989	0.989	0.989
All	IDS18	0.988	0.988	0.988	0.988

In this table, some combinations of train-test sets generate higher metric values than others. Based on F1 and accuracy (which are highly correlated—their correlation equals 0.99), one may observe that classifiers trained on ToN perform pretty well when tested on IDS18 and NB15 (obviously, it obtained the highest score for ToN-origin test set). In contrast, its performance on the BoT test set is dramatically low. Similar patterns may be seen when training on NB15 and IDS18—also, the testing on BoT results in measures close to 0. High

metrics values imply relatively high similarity of the network traffic between sets NB15 and IDS18 and lower similarity between each of them and the ToN. It may imply that they may be considered generalizable. The BoT set is, however, a different case, as indicated by the low values of measures. When, in turn, this set is used as a training one, the measures obtained when the remainder of sets as test ones are remarkably lower (F1: 0.584,0.380) or even close to 0 (0.050). This behavior confirms that the BoT differs from other sets and is poorly generalizable.

To dive deeper into this issue, one must look into all the sets, performing quantitative and qualitative investigations. Looking at Table 1, one may see that the principal difference between BoT and the remaining sets, apart from the origin of data and classes, relies on the class balance. In all but BoT datasets, the *Benign* traffic class dominates, in IDS18 and NB15, it dominates totally (88.05% and 96.02%, respectively), while in ToN, it is still the most frequent class, gathering 36.01 % of samples. This is because the share of classes in the dataset tries to reflect the one in the typical network flow, where intrusions and attacks are relatively rare and what dominates is the benign network traffic. From this point of view, BoT looks completely different; the share of the *Benign* class is less than 1%, and the set is almost equally divided into two harmful classes: *DoS* and *DDoS*. All of the above raises suspicion that the high values of metrics obtained in the cross-validation of IDS18, ToN, and NB15 are caused by the significant share of the benign traffic in the datasets.

To validate this hypothesis, the binary classification setup is applied, focusing on the *Benign* traffic class and measuring the ability of classifiers to correctly classify samples of this class (true-case), treating the rest of the classes as the false-case. Table 5 shows the classification metrics in this setup. Following the previous considerations, when analyzing the *Benign* traffic class, we focus on precision and false positive rate (FPR) measures, which are the most adequate in this case. The F1 measure is used as the supplementary one.

Table 5. Binary classification results—class *Benign*

Train	Test	A	P	R	F1	FNR	FPR
BoT	BoT	1.000	0.956	0.988	0.972	0.012	0.000
BoT	ToN	0.536	0.209	0.104	0.139	0.896	0.221
BoT	NB15	0.271	0.970	0.248	0.395	0.752	0.188
BoT	IDS18	0.548	0.963	0.507	0.663	0.493	0.146
ToN	BoT	0.190	0.005	0.974	0.009	0.026	0.813
ToN	ToN	0.984	0.967	0.990	0.978	0.001	0.019
ToN	NB15	0.897	0.959	0.934	0.946	0.067	0.975
ToN	IDS18	0.793	0.873	0.895	0.884	0.105	0.957
NB15	BoT	0.468	0.006	0.922	0.012	0.078	0.534
NB15	ToN	0.351	0.354	0.971	0.518	0.029	0.999
NB15	NB15	0.970	0.999	0.998	0.998	0.002	0.037
NB15	IDS18	0.843	0.876	0.957	0.915	0.043	0.997
IDS18	BoT	0.041	0.003	0.867	0.006	0.134	0.962
IDS18	ToN	0.294	0.304	0.743	0.431	0.257	0.959
IDS18	NB15	0.958	0.960	0.998	0.979	0.003	0.999
IDS18	IDS18	0.992	0.995	0.996	0.996	0.004	0.036
All	BoT	0.962	0.080	0.930	0.148	0.071	0.038
All	ToN	0.982	0.960	0.990	0.975	0.010	0.023
All	NB15	0.997	0.999	0.998	0.998	0.002	0.040
All	IDS18	0.989	0.994	0.993	0.994	0.007	0.045

The investigation confirms the influence of the *Benign* class on the multi-class classification results. The high precision (P) and low false positive rate (FPR) values are present for the same train and test set combinations as in the multi-class case. In particular, the IDS18 and NB15 datasets are very close to one another regarding cross-validated classification results. It implies the similarity of samples from both sets belonging to the *Benign* traffic class. The ToN dataset performs worse; however, the results of the C class are lower than in the multi-class case. This property may come from differences in the size of the *Benign* class. The number of samples for IDS18 and NB15 is approximately 2.5 times higher than in the ToN case (see Table 1). Looking, in turn, on the BoT dataset, one may observe dramatically weak results.

However, not only qualities make the difference. In the case of NB15 and IDS18, the captured benign traffic consists of the natural transaction data based on standard application protocols, like HTTP, HTTPS, FTP, SSH, and email. In both cases, such a user's normal behavior profile was selected. The BoT data set, however, was constructed to mimic the usage of adversarial behavior of compromised IoT networks as Botnet malware. Therefore the regular traffic is substantially reduced. One may notice that the *Benign* class network traffic in the BoT consists only of a realistic smart home network, as the five IoT devices. Several instances are relatively small and do not include the normal user's behavior like in the NB15 and IDS18 datasets [6].

The t-SNE visualization of *Benign* class is shown in Figure 2. The area covered by samples belonging to particular datasets overlaps. Observation of distributions of points in reduced 2D space confirms previous conclusions—areas covered by IDS18 and NB15 are very close to one another—practically, they fully overlap; the ToN dataset only partially overlaps both. Observing, in turn, the distribution of BoT 2D points, one immediately finds that they are far apart from the others. Less intensive point cloud reflects the difference in the number of samples (approx 100 times less than ToN and 250 times less than NB15 and IDS18).

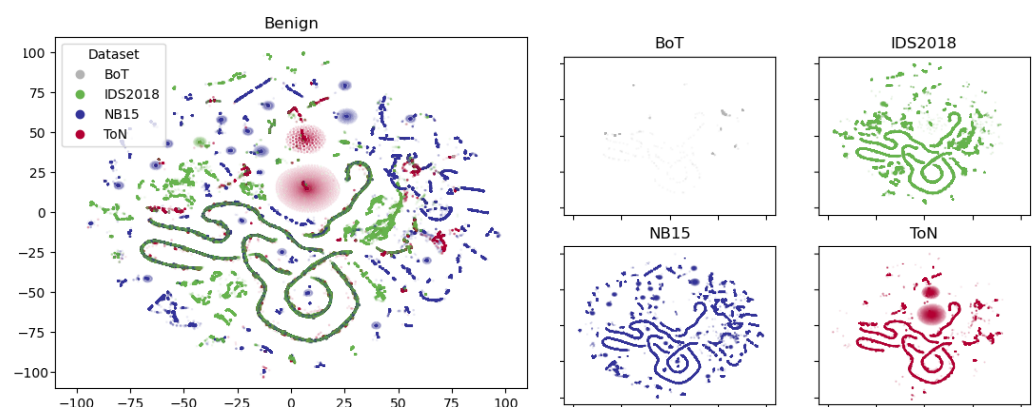


Figure 2. The t-SNE visualization of benign traffic class with labeled original sets and their share in class samples ((left)—all classes in a single chart; (right)—separated charts per class).

5.2. Harmful Traffic Classes Detection

The first phase of experiments revealed that the relatively good results that may confirm the generalizability of three of four datasets are mostly driven by the good classification of the *Benign* class that dominates over other classes and consists of samples that exhibits some intra-dataset resemblance. In the case of the BoT dataset, even multi-class measures do not confirm the generalizability.

A *Benign* class in the network traffic dataset is necessary to properly train the ML models to detect the remaining classes referring to harmful traffic, finally allowing them to distinguish between regular (benign) traffic and various dangerous ones. From this point of view, correctly detecting *Benign* traffic by models trained on datasets other than the one on which the model is tested would eventually allow for separating benign from harmful traffic. One need, however, to consider two additional issues. At first, it is restricted to the NB15 and IDS18 datasets, containing the enterprise network traffic. In the case of the IoT traffic, results are not that optimistic. Secondly, in most cases, the primary goal of NIDS exceeds simple differentiation between benign and harmful classes—the system should also detect the type of danger. In other words, it should correctly classify harmful data samples into one of the pre-defined classes.

To investigate this issue, we conducted experiments similar to the one with *Benign* class binary classification but on classes representing harmful traffic. Such experiments should be conducted on co-occurring classes, i.e., classes in more than one dataset. Looking at Table 1, one may easily observe that the collection of such classes is relatively small. It consists of just four classes: *DoS*, *DDoS*, *Reconnaissance*, and *Backdoor*.

All the co-occurring classes are analyzed using the previously mentioned measures. However, following again our previous considerations, in this case, we focus on the Recall measure, which should converge to 1 when the performance of classifiers grows, and the false negative rate (FNR) should converge to 0. The measures obtained for all co-occurring harmful classes are shown in Table 6.

The only harmful traffic class in all four sets is *Denial of Service (DoS)*. It is one of two principal classes of the BoT dataset, so it is no wonder that the number of samples belonging to this set far exceeds the number of samples in the remainder of the sets. It is more than ten times higher than in the second most frequent class—ToN. This situation resembles the case of the *Benign* class—some datasets contain more samples of a particular class than others. Nevertheless, contrary to the *Benign* class case, this does not directly imply that the classifier trained on these sets performs well on others, which may be observed in metrics values. Here, despite high values of precision, recall, and F1 measures and low value of FNR for the case BoT-BoT (train-test), the model trained on BoT and tested on three other datasets perform poorly—the metric values are close to 0, while error value (FNR) is close to 1. It is because samples of the *DoS* class from different original datasets occupy disjoint regions in the feature space. It is visible on the t-SNE visualization shown in Figure 3 (left). For the same reason, other cross-validated combinations of train-test sets perform poorly. To understand why this happens, one needs to look carefully at the characteristics of the *DoS* class, which, in general, is differentiated in particular sets. These datasets differ mainly in the types of systems targeted (enterprise networks vs. IoT) and the techniques employed in the attacks. In NB15, the DoS attacks involve traditional network-based denial-of-service attempts targeting system resources, typically from a single host. In IDS18, attacks are more varied, including different types of flooding (e.g., HTTP, SYN flood) that aim to disrupt services, targeting more modern network environments. BoT attacks focus on disrupting IoT devices' functionality and exploiting IoT ecosystems' limited resources, with traffic aimed at overwhelming low-power devices. Finally, ToN attacks target IoT devices and traditional networks, showcasing resource exhaustion in IoT environments while illustrating cloud-based and industrial infrastructure vulnerabilities.

Table 6. Binary classification results harmful classes (Do—DoS, DD—DDoS, Ba—Backdoor, Re—Reconnaissance).

Class	Train	Test	A	P	R	F1	FNR	FPR
Do	BoT	BoT	0.986	0.984	0.984	0.984	0.017	0.013
Do	BoT	ToN	0.954	0.000	0.000	0.000	1.000	0.004
Do	BoT	NB15	0.937	0.024	0.072	0.019	0.928	0.061
Do	BoT	IDS18	0.972	0.003	0.000	0.000	1.000	0.003
Do	ToN	BoT	0.559	0.000	0.000	0.000	1.000	0.000
Do	ToN	ToN	0.990	0.884	0.884	0.884	0.116	0.005
Do	ToN	NB15	0.993	0.000	0.000	0.000	1.000	0.005
Do	ToN	IDS18	0.946	0.000	0.000	0.000	1.000	0.030
Do	NB15	BoT	0.558	0.002	0.000	0.000	1.000	0.000
Do	NB15	ToN	0.957	0.150	0.007	0.013	0.993	0.002
Do	NB15	NB15	0.997	0.352	0.332	0.341	0.669	0.002
Do	NB15	IDS18	0.960	0.000	0.000	0.000	1.000	0.015
Do	IDS18	BoT	0.559	0.185	0.000	0.000	1.000	0.000
Do	IDS18	ToN	0.944	0.000	0.000	0.000	1.000	0.014
Do	IDS18	NB15	0.997	0.000	0.000	0.000	1.000	0.000
Do	IDS18	IDS18	1.000	1.000	1.000	1.000	0.000	0.000
Do	All	BoT	0.986	0.984	0.984	0.984	0.017	0.013
Do	All	ToN	0.990	0.886	0.863	0.874	0.137	0.005
Do	All	NB15	0.997	0.345	0.319	0.331	0.682	0.002
Do	All	IDS18	1.000	0.962	1.000	0.980	0.000	0.002
DD	BoT	BoT	0.995	0.994	0.994	0.994	0.006	0.006
DD	BoT	ToN	0.880	0.000	0.000	0.000	1.000	0.000
DD	BoT	IDS18	0.927	0.814	0.001	0.003	0.999	0.000
DD	ToN	BoT	0.514	0.000	0.000	0.000	1.000	0.001
DD	ToN	ToN	0.991	0.952	0.975	0.963	0.025	0.007
DD	ToN	IDS18	0.924	0.000	0.000	0.000	1.000	0.003
DD	IDS18	BoT	0.514	0.000	0.000	0.000	1.000	0.000
DD	IDS18	ToN	0.880	0.238	0.000	0.001	1.000	0.000
DD	IDS18	IDS18	1.000	0.998	1.000	0.999	0.000	0.000
DD	All	BoT	0.994	0.994	0.994	0.994	0.006	0.006
DD	All	ToN	0.991	0.952	0.975	0.963	0.025	0.007
DD	All	IDS18	1.000	0.997	1.000	0.999	0.000	0.000
Ba	ToN	ToN	1.000	0.994	0.992	0.993	0.008	0.000
Ba	ToN	NB15	0.999	0.000	0.000	0.000	1.000	0.000
Ba	NB15	ToN	0.999	0.000	0.000	0.000	1.000	0.000
Ba	NB15	NB15	0.998	0.150	0.158	0.154	0.842	0.001
Ba	All	ToN	1.000	0.999	0.987	0.993	0.013	0.000
Ba	All	NB15	0.998	0.153	0.159	0.156	0.841	0.001
Re	BoT	BoT	0.991	0.935	0.934	0.934	0.066	0.005
Re	BoT	NB15	0.314	0.007	0.893	0.014	0.107	0.689
Re	NB15	BoT	0.931	0.500	0.000	0.000	1.000	0.000
Re	NB15	NB15	0.998	0.858	0.771	0.812	0.229	0.001
Re	All	BoT	0.953	0.855	0.381	0.527	0.619	0.005
Re	All	NB15	0.998	0.859	0.771	0.813	0.229	0.001

Another frequent class is the Distributed Denial of Service (*DDoS*) class in three datasets: BoT, ToN, and IDS18. Here also, BoT keeps most of the samples belonging to this class. Also, careful analysis, following that performed in the *DoS* class (see Figure 3,

right, for t-SNE visualization) leads to the conclusion that the level of generalization of this class is extremely low—measures for all but own combination of datasets are close to 0, while FNR error rates—close to 1. The main differences lie in the attack targets: IDS18 focuses on traditional networks, while BoT and ToN deal with IoT environments, with ToN, also incorporating industrial and cloud systems in its scope. In IDS18, attacks involve large-scale, distributed attacks using botnets targeting traditional network services. In BoT, attacks specifically exploit IoT devices, using compromised IoT devices (botnets) to generate high volumes of traffic aimed at overloading the network or devices. In ToN, attacks target IoT and traditional systems, reflecting complex, mixed environments, including cloud and edge infrastructure.

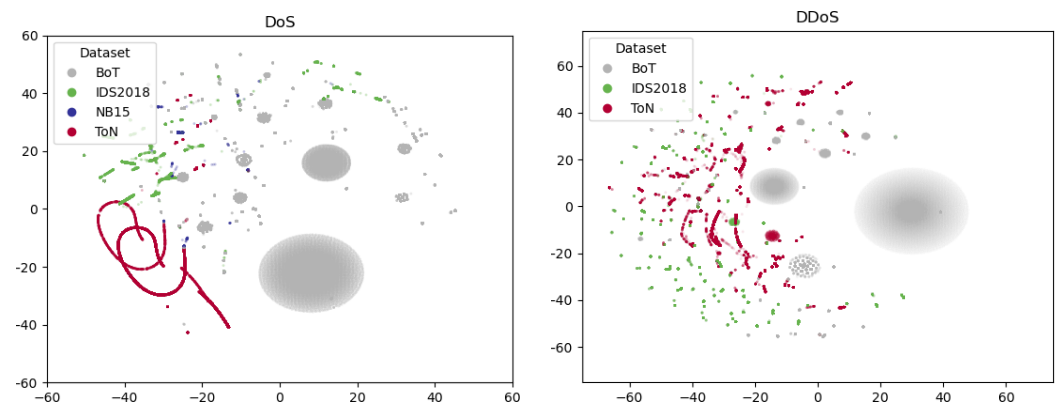


Figure 3. The t-SNE visualization of *DoS* class (left) and *DDoS* class (right) with labeled original sets.

It is worth noting that the detection of DoS and DDoS attacks is based on a filtered set of traffic features. This ultimate set does not include the source IP address. In many attacks, the huge repeatability of the IP source is the principal indicator of this type of attack. However, hopefully not the only one, other determining factors are usually present in higher layers of the protocol, which influence other traffic features in our datasets.

The last two co-occurring classes are *Backdoor* and *Reconnaissance*. Both are present in just two sets and are much less frequent than *Benign*, *DoS*, and *DDoS*. In each of the two classes, analyzing the metrics and visualizations (Figure 4) and applying the same train of thought, one quickly notices that, although samples from different sets are assigned to a class of the same name, the internal characteristics of features exhibits significant differences between features of traffic samples of the same class originated from different datasets. The main difference between the *Backdoor* class in ToN and NB15 is the environment being targeted. The ToN focuses on IoT devices and industrial systems, allowing attackers persistent, unauthorized access to critical infrastructure, including smart devices and edge systems. In the NB15, *Backdoor* attacks target traditional network systems and servers, aiming to maintain covert access to corporate or personal computing environments. Similarly, the difference between *Reconnaissance* class in BoT and NB15 is the type of targets. The BoT focuses on probing and scanning IoT devices to find vulnerabilities specific to resource-constrained, internet-connected devices. In contrast, in NB15, the traditional network infrastructure is scanned, gathering information such as open ports or system configurations in more conventional IT environments.

Apart from models trained on particular datasets, the models trained on a superposition of datasets were also investigated. The training set was created by merging all four component datasets into a single superset of 4 million samples. The performance of such a classifier was measured in the same way as in individual cases—the resulting measures are shown in the last section of each table (the train set name is 'All' in this case). Contrary to models trained on individual sets, the model trained on superset performs well on all

test sets. The reason for such good results is the increase of intra-class variability achieved thanks to merging. This observation also indicates that extending the training set during the use of the model, followed by re-training, allows for adaptation of the NIDS to the real working environment.

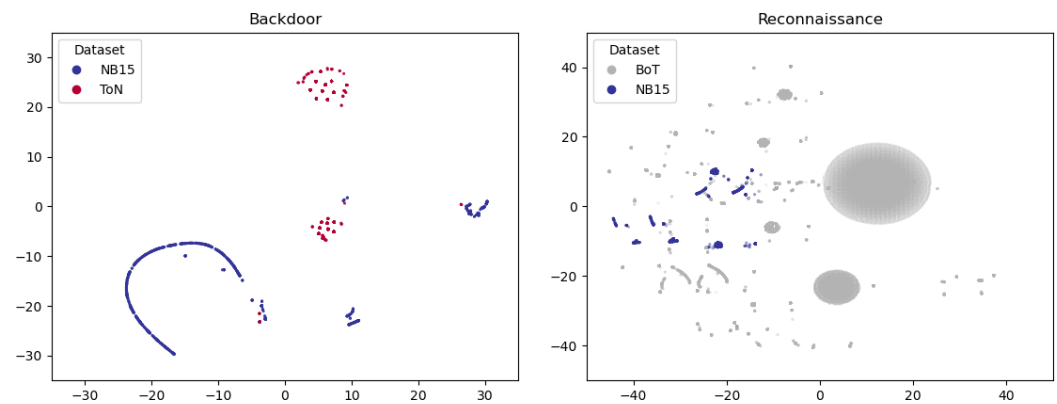


Figure 4. The t-SNE visualization of *backdoor* class (left) and Reconnaissance (right) with labeled original sets.

6. Conclusions

Our work aimed to investigate the generalizability of datasets used to train the ML models used in NIDS, which is a crucial issue, given possible practical implementations of these models to classify the network traffic. The results obtained allow us to formulate the following conclusions:

1. In the case of diverse traffic datasets containing an essential amount of *Benign* traffic class samples, the multi-class classification results are strongly biased by this class. Relatively high measures obtained in the multi-class case are caused by efficient identification of normal traffic, which does not happen in the case of the harmful one. Consequently, such models may correctly detect *Benign* classes but cannot distinguish between various harmful ones.
2. Contrary to the *Benign* class that shows some similarities between different datasets, the harmful classes differ depending on the dataset they originate from. This means that models trained on a particular dataset are hardly generalizable to other data in detecting and classifying harmful traffic.
3. The exact names of harmful classes in different sets, or—more generally—in different data sources, do not necessarily imply that the nature of sources of this kind of traffic is similar. On the other hand, classes bearing different names in different data sources may represent traffic of similar nature.
4. The variety of network traffic variants results in various classes and possible feature value combinations in samples of the traffic data used in the NIDS systems. The available datasets contain only a part of a spectrum of possible traffic variants. Nevertheless, they are widely used to train new models intended to work with another traffic variant that is not necessarily similar, which has been reported in many papers. The outcomes of our research show that one always needs to carefully analyze similarities between the fundamental nature of harmful classes present in the dataset used and properties of possible harmful traffic that may occur in the target environment. Only high similarity may eventually result in correct classification in the target environment.
5. Combining datasets collecting the traffic data captured in differentiated environments results in more exhaustive and robust collections of traffic samples. Finally, it may result in ML models used in NIDS that are ready to deal with more diverse traffic.

Author Contributions: Conceptualization, M.I.; methodology, M.I., W.G. and D.O.; software, J.K.; validation, M.I., J.K. and F.P.; formal analysis, M.I.; investigation, M.I., D.O. and J.K.; resources, J.K.; data curation, J.K.; writing—original draft preparation, M.I.; writing—review and editing, M.I., D.O., W.G., J.K. and F.P.; visualization, M.I., D.O.; supervision, M.I.; project administration, M.I. and W.G.; funding acquisition, W.G. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: The research described in this paper was done using publicly available datasets. They are available via the web at addresses provided by their authors, which are to be found in the appropriate papers.

Conflicts of Interest: The authors declare no conflicts of interest.

Abbreviations

The following abbreviations are used in this manuscript:

NIDS	Network Intrusion Detection System
ML	Machine Learning
IoT	Internet of Things
IIoT	Industrial Internet of Things
NB15	UNSW-NB15 dataset in ‘v2’ version
IDS18	IC-CSE-IDS2018 dataset in ‘v2’ version
BoT	BoT-IoT dataset in ‘v2’ version
ToN	ToN-IoT dataset in ‘v2’ version
Extra Tree	Extremely Randomized Tree
SNE	Stochastic Neighbor Embedding
t-SNE	t-Distributed Stochastic Neighbor Embedding

References

1. Mahbub, M. Progressive researches on IoT security: An exhaustive analysis from the perspective of protocols, vulnerabilities, and preemptive architectonics. *J. Netw. Comput. Appl.* **2020**, *168*, 102761. [\[CrossRef\]](#)
2. Sommer, R.; Paxson, V. Outside the Closed World: On Using Machine Learning for Network Intrusion Detection. In Proceedings of the 2010 IEEE Symposium on Security and Privacy, Oakland, CA, USA, 16–19 May 2010; pp. 305–316. [\[CrossRef\]](#)
3. Kheddar, H.; Himeur, Y.; Awad, A.I. Deep transfer learning for intrusion detection in industrial control networks: A comprehensive review. *J. Netw. Comput. Appl.* **2023**, *220*, 103760. [\[CrossRef\]](#)
4. Moustafa, N.; Slay, J. UNSW-NB15: A comprehensive data set for network intrusion detection systems. In Proceedings of the Military Communications and Information Systems Conference (MilCIS), Canberra, Australia, 16–18 November 2015.
5. Sharafaldin, I.; Lashkari, A.H.; Ghorbani, A.A. Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization. In Proceedings of the International Conference on Information Systems Security and Privacy, Beijing, China, 4–6 September 2018.
6. Koroniotis, N.; Moustafa, N.; Sitnikova, E.; Turnbull, B. Towards the development of realistic botnet dataset in the internet of things for network forensic analytics: Bot-iot dataset. *Future Gener. Comput. Syst.* **2019**, *100*, 779–796. [\[CrossRef\]](#)
7. Alsaedi, A.; Moustafa, N.; Tari, Z.; Mahmood, A.; Anwar, A. TON_IoT Telemetry Dataset: A New Generation Dataset of IoT and IIoT for Data-Driven Intrusion Detection Systems. *IEEE Access* **2020**, *8*, 165130–165150. [\[CrossRef\]](#)
8. Sarhan, M.; Layeghy, S.; Portmann, M. Towards a standard feature set for network intrusion detection system datasets. *Mob. Netw. Appl.* **2022**, *27*, 357–370. [\[CrossRef\]](#)
9. Geurts, P.; Ernst, D.; Wehenkel, L. Extremely randomized trees. *Mach. Learn.* **2006**, *63*, 3–42. [\[CrossRef\]](#)
10. van der Maaten, L.; Hinton, G. Visualizing Data using t-SNE. *J. Mach. Learn. Res.* **2008**, *9*, 2579–2605.
11. Khraisat, A.; Gondal, I.; Vamplew, P.; Kamruzzaman, J. Survey of intrusion detection systems: Techniques, datasets and challenges. *Cybersecurity* **2019**, *2*, 20. [\[CrossRef\]](#)

12. Azab, A.; Khasawneh, M.; Alrabaee, S.; Choo, K.K.R.; Sarsour, M. Network traffic classification: Techniques, datasets, and challenges. *Digit. Commun. Netw.* **2022**, *10*, 676–692. [\[CrossRef\]](#)
13. Ali, T.E.; Chong, Y.W.; Manickam, S. Machine Learning Techniques to Detect a DDoS Attack in SDN: A Systematic Review. *Appl. Sci.* **2023**, *13*, 3183. [\[CrossRef\]](#)
14. Dini, P.; Elhanashi, A.; Begni, A.; Saponara, S.; Zheng, Q.; Gasmi, K. Overview on Intrusion Detection Systems Design Exploiting Machine Learning for Networking Cybersecurity. *Appl. Sci.* **2023**, *13*, 7507. [\[CrossRef\]](#)
15. Liu, H.; Lang, B. Machine Learning and Deep Learning Methods for Intrusion Detection Systems: A Survey. *Appl. Sci.* **2019**, *9*, 4396. [\[CrossRef\]](#)
16. Martínez Torres, J.; Iglesias Comesaña, C.; García-Nieto, P.J. Review: Machine learning techniques applied to cybersecurity. *Int. J. Mach. Learn. Cybern.* **2019**, *10*, 2823–2836. [\[CrossRef\]](#)
17. Sarker, I.H.; Kayes, A.S.M.; Badsha, S.; Alqahtani, H.; Watters, P.; Ng, A. Cybersecurity data science: An overview from machine learning perspective. *J. Big Data* **2020**, *7*, 41. [\[CrossRef\]](#)
18. Bhuyan, M.H.; Bhattacharyya, D.K.; Kalita, J.K. Network Anomaly Detection: Methods, Systems and Tools. *IEEE Commun. Surv. Tutorials* **2014**, *16*, 303–336. [\[CrossRef\]](#)
19. Dong, S.; Xia, Y.; Peng, T. Network Abnormal Traffic Detection Model Based on Semi-Supervised Deep Reinforcement Learning. *IEEE Trans. Netw. Serv. Manag.* **2021**, *18*, 4197–4212. [\[CrossRef\]](#)
20. Ahmad, R.; Alsmadi, I.; Alhamdani, W.; Tawalbeh, L. Models versus Datasets: Reducing Bias through Building a Comprehensive IDS Benchmark. *Future Internet* **2021**, *13*, 318. [\[CrossRef\]](#)
21. Sičić, I.; Petrović, N.; Slovenec, K.; Mikuc, M. Evaluation of Inter-Dataset Generalisability of Autoencoders for Network Intrusion Detection. In Proceedings of the 2023 17th International Conference on Telecommunications (ConTEL), Tbilisi, Georgia, 19–21 June 2023; pp. 1–7. [\[CrossRef\]](#)
22. Al-Riyami, S.; Lisitsa, A.; Coenen, F. Cross-Datasets Evaluation of Machine Learning Models for Intrusion Detection Systems. In *Proceedings of the Sixth International Congress on Information and Communication Technology, Chongqing, China, 14–16 October 2022*; Yang, X.S., Sherratt, S., Dey, N., Joshi, A., Eds.; Springer: Singapore, 2022; pp. 815–828.
23. Sarhan, M.; Layeghy, S.; Portmann, M. Evaluating Standard Feature Sets Towards Increased Generalisability and Explainability of ML-Based Network Intrusion Detection. *Big Data Res.* **2022**, *30*, 100359. [\[CrossRef\]](#)
24. Layeghy, S.; Portmann, M. Explainable Cross-domain Evaluation of ML-based Network Intrusion Detection Systems. *Comput. Electr. Eng.* **2023**, *108*, 108692. [\[CrossRef\]](#)
25. Layeghy, S.; Gallagher, M.; Portmann, M. Benchmarking the benchmark—Comparing synthetic and real-world Network IDS datasets. *J. Inf. Secur. Appl.* **2024**, *80*, 103689. [\[CrossRef\]](#)
26. D’hooge, L.; Wauters, T.; Volckaert, B.; De Turck, F. Inter-dataset generalization strength of supervised machine learning methods for intrusion detection. *J. Inf. Secur. Appl.* **2020**, *54*, 102564. [\[CrossRef\]](#)
27. Cantone, M.; Marrocco, C.; Bria, A. On the Cross-Dataset Generalization of Machine Learning for Network Intrusion Detection. *arXiv* **2024**, arXiv:2402.10974. [\[CrossRef\]](#)
28. Kayacik, H.; Zincir-Heywood, A.; Heywood, M. On dataset biases in a learning system with minimum a priori information for intrusion detection. In Proceedings of the Second Annual Conference on Communication Networks and Services Research, Athens, Greece, 22–24 September 2004; pp. 181–189. [\[CrossRef\]](#)
29. Bouke, M.A.; Abdullah, A. An empirical study of pattern leakage impact during data preprocessing on machine learning-based intrusion detection models reliability. *Expert Syst. Appl.* **2023**, *230*, 120715. [\[CrossRef\]](#)
30. da Silva Ruffo, V.G.; Brandão Lent, D.M.; Komarchesqui, M.; Schiavon, V.F.; de Assis, M.V.O.; Carvalho, L.F.; Proença, M.L. Anomaly and intrusion detection using deep learning for software-defined networks: A survey. *Expert Syst. Appl.* **2024**, *256*, 124982. [\[CrossRef\]](#)
31. Ring, M.; Wunderlich, S.; Scheuring, D.; Landes, D.; Hotho, A. A survey of network-based intrusion detection data sets. *Comput. Secur.* **2019**, *86*, 147–167. [\[CrossRef\]](#)
32. Komisarek, M.; Pawlicki, M.; Kozik, R.; Hołubowicz, W.; Choraś, M. How to Effectively Collect and Process Network Data for Intrusion Detection? *Entropy* **2021**, *23*, 1532. [\[CrossRef\]](#)
33. Guerra, J.L.; Catania, C.; Veas, E. Datasets are not enough: Challenges in labeling network traffic. *Comput. Secur.* **2022**, *120*, 102810. [\[CrossRef\]](#)
34. Bönninghausen, P.; Uetz, R.; Henze, M. Introducing a Comprehensive, Continuous, and Collaborative Survey of Intrusion Detection Datasets. In Proceedings of the 17th Cyber Security Experimentation and Test Workshop, Philadelphia, PA, USA, 13 August 2024; pp. 34–40. [\[CrossRef\]](#)
35. Qi, L.; Yang, Y.; Zhou, X.; Rafique, W.; Ma, J. Fast Anomaly Identification Based on Multiaspect Data Streams for Intelligent Intrusion Detection Toward Secure Industry 4.0. *IEEE Trans. Ind. Inform.* **2022**, *18*, 6503–6511. [\[CrossRef\]](#)
36. Zhou, X.; Hu, Y.; Liang, W.; Ma, J.; Jin, Q. Variational LSTM Enhanced Anomaly Detection for Industrial Big Data. *IEEE Trans. Ind. Inform.* **2021**, *17*, 3469–3477. [\[CrossRef\]](#)

37. Chen, D.; Zhang, F.; Zhang, X. Heterogeneous IoT Intrusion Detection Based on Fusion Word Embedding Deep Transfer Learning. *IEEE Trans. Ind. Inform.* **2023**, *19*, 9183–9193. [\[CrossRef\]](#)
38. Telikani, A.; Rudbardeh, N.E.; Soleymannpour, S.; Shahbahrami, A.; Shen, J.; Gaydadjiev, G.; Hassanpour, R. A Cost-Sensitive Machine Learning Model with Multitask Learning for Intrusion Detection in IoT. *IEEE Trans. Ind. Inform.* **2024**, *20*, 3880–3890. [\[CrossRef\]](#)
39. Xiong, S.H.; Qiu, M.R.; Li, G.; Zhang, H.; Chen, Z.S. Balancing the signals: Bayesian equilibrium selection for high-speed railway sensor defense. *Inf. Sci.* **2024**, *661*, 120196. [\[CrossRef\]](#)
40. Zeeshan, M.; Riaz, Q.; Bilal, M.A.; Shahzad, M.K.; Jabeen, H.; Haider, S.A.; Rahim, A. Protocol-Based Deep Intrusion Detection for DoS and DDoS Attacks Using UNSW-NB15 and Bot-IoT Data-Sets. *IEEE Access* **2022**, *10*, 2269–2283. [\[CrossRef\]](#)
41. Wang, X.; Wang, X.; He, M.; Zhang, M.; Lu, Z. Spatial-Temporal Graph Model Based on Attention Mechanism for Anomalous IoT Intrusion Detection. *IEEE Trans. Ind. Inform.* **2024**, *20*, 3497–3509. [\[CrossRef\]](#)
42. Basati, A.; Faghih, M.M. PDAE: Efficient network intrusion detection in IoT using parallel deep auto-encoders. *Inf. Sci.* **2022**, *598*, 57–74. [\[CrossRef\]](#)
43. Zhang, L.; Xie, X.; Xiao, K.; Bai, W.; Liu, K.; Dong, P. MANomaly: Mutual adversarial networks for semi-supervised anomaly detection. *Inf. Sci.* **2022**, *611*, 65–80. [\[CrossRef\]](#)
44. Khan, M.A.; Iqbal, N.; Imran, Jamil, H.; Kim, D.H. An optimized ensemble prediction model using AutoML based on soft voting classifier for network intrusion detection. *J. Netw. Comput. Appl.* **2023**, *212*, 103560. [\[CrossRef\]](#)
45. Cao, B.; Li, C.; Song, Y.; Qin, Y.; Chen, C. Network Intrusion Detection Model Based on CNN and GRU. *Appl. Sci.* **2022**, *12*, 4184. [\[CrossRef\]](#)
46. Zhang, J.; Luo, C.; Carpenter, M.; Min, G. Federated Learning for Distributed IIoT Intrusion Detection Using Transfer Approaches. *IEEE Trans. Ind. Inform.* **2023**, *19*, 8159–8169. [\[CrossRef\]](#)
47. Long, J.; Liang, W.; Li, K.C.; Wei, Y.; Marino, M.D. A Regularized Cross-Layer Ladder Network for Intrusion Detection in Industrial Internet of Things. *IEEE Trans. Ind. Inform.* **2023**, *19*, 1747–1755. [\[CrossRef\]](#)
48. Vo, H.V.; Du, H.P.; Nguyen, H.N. AI-powered intrusion detection in large-scale traffic networks based on flow sensing strategy and parallel deep analysis. *J. Netw. Comput. Appl.* **2023**, *220*, 103735. [\[CrossRef\]](#)
49. Yue, M.; Yan, H.; Han, R.; Wu, Z. A DDoS attack detection method based on IQR and DFFCNN in SDN. *J. Netw. Comput. Appl.* **2025**, *240*, 104203. [\[CrossRef\]](#)
50. Najafi Mohsenabad, H.; Tut, M.A. Optimizing Cybersecurity Attack Detection in Computer Networks: A Comparative Analysis of Bio-Inspired Optimization Algorithms Using the CSE-CIC-IDS 2018 Dataset. *Appl. Sci.* **2024**, *14*, 1044. [\[CrossRef\]](#)
51. Leevy, J.L.; Khoshgoftaar, T.M. A survey and analysis of intrusion detection models based on CSE-CIC-IDS2018 Big Data. *J. Big Data* **2020**, *7*, 104. [\[CrossRef\]](#)
52. Latif, S.; Huma, Z.e.; Jamal, S.S.; Ahmed, F.; Ahmad, J.; Zahid, A.; Dashtipour, K.; Aftab, M.U.; Ahmad, M.; Abbasi, Q.H. Intrusion Detection Framework for the Internet of Things Using a Dense Random Neural Network. *IEEE Trans. Ind. Inform.* **2022**, *18*, 6435–6444. [\[CrossRef\]](#)
53. Ding, W.; Abdel-Basset, M.; Mohamed, R. DeepAK-IoT: An effective deep learning model for cyberattack detection in IoT networks. *Inf. Sci.* **2023**, *634*, 157–171. [\[CrossRef\]](#)
54. Kumar, P.; Kumar, R.; Gupta, G.P.; Tripathi, R.; Srivastava, G. P2TIF: A Blockchain and Deep Learning Framework for Privacy-Preserved Threat Intelligence in Industrial IoT. *IEEE Trans. Ind. Inform.* **2022**, *18*, 6358–6367. [\[CrossRef\]](#)
55. Ruzafa-Alcázar, P.; Fernández-Saura, P.; Mármol-Campos, E.; González-Vidal, A.; Hernández-Ramos, J.L.; Bernal-Bernabe, J.; Skarmeta, A.F. Intrusion Detection Based on Privacy-Preserving Federated Learning for the Industrial IoT. *IEEE Trans. Ind. Inform.* **2023**, *19*, 1145–1154. [\[CrossRef\]](#)
56. Booi, T.M.; Chiscop, I.; Meeuwissen, E.; Moustafa, N.; den Hartog, F.T.H. ToN IoT: The Role of Heterogeneity and the Need for Standardization of Features and Attack Types in IoT Network Intrusion Data Sets. *IEEE Internet Things J.* **2022**, *9*, 485–496. [\[CrossRef\]](#)
57. Kumar, R.; Kumar, P.; Tripathi, R.; Gupta, G.P.; Islam, A.K.M.N.; Shorfuzzaman, M. Permissioned Blockchain and Deep Learning for Secure and Efficient Data Sharing in Industrial Healthcare Systems. *IEEE Trans. Ind. Inform.* **2022**, *18*, 8065–8073. [\[CrossRef\]](#)
58. Qiao, H.; Novikov, B.; Blech, J.O. Concept Drift Analysis by Dynamic Residual Projection for Effectively Detecting Botnet Cyber-Attacks in IoT Scenarios. *IEEE Trans. Ind. Inform.* **2022**, *18*, 3692–3701. [\[CrossRef\]](#)
59. Al-Taleb, N.; Saqib, N.A. Towards a Hybrid Machine Learning Model for Intelligent Cyber Threat Identification in Smart City Environments. *Appl. Sci.* **2022**, *12*, 1863. [\[CrossRef\]](#)
60. Sadhwani, S.; Manibalan, B.; Muthalagu, R.; Pawar, P. A Lightweight Model for DDoS Attack Detection Using Machine Learning Techniques. *Appl. Sci.* **2023**, *13*, 9937. [\[CrossRef\]](#)
61. Tareq, I.; Elbagoury, B.M.; El-Regaily, S.; El-Horbaty, E.S.M. Analysis of ToN-IoT, UNW-NB15, and Edge-IIoT Datasets Using DL in Cybersecurity for IoT. *Appl. Sci.* **2022**, *12*, 9572. [\[CrossRef\]](#)

62. Al Hwaitat, A.K.; Fakhouri, H.N. Adaptive Cybersecurity Neural Networks: An Evolutionary Approach for Enhanced Attack Detection and Classification. *Appl. Sci.* **2024**, *14*, 9142. [\[CrossRef\]](#)
63. Fu, J.; Wang, L.; Ke, J.; Yang, K.; Yu, R. TSIDS: Spatial-temporal fusion gating Multilayer Perceptron for network intrusion detection. *Expert Syst. Appl.* **2025**, *263*, 125687. [\[CrossRef\]](#)
64. Nguyen, V.Q.; Nguyen, V.H.; Ngo, L.T.; Nguyen, L.M.; Le-Khac, N.A. Variational Deep Clustering approaches for anomaly-based cyber-attack detection. *J. Netw. Comput. Appl.* **2025**, *240*, 104182. [\[CrossRef\]](#)
65. Tahaei, H.; Afifi, F.; Asemi, A.; Zaki, F.; Anuar, N.B. The rise of traffic classification in IoT networks: A survey. *J. Netw. Comput. Appl.* **2020**, *154*, 102538. [\[CrossRef\]](#)
66. Lee, S.W.; Mohammed sidqi, H.; Mohammadi, M.; Rashidi, S.; Rahmani, A.M.; Masdari, M.; Hosseinzadeh, M. Towards secure intrusion detection systems using deep learning techniques: Comprehensive analysis and review. *J. Netw. Comput. Appl.* **2021**, *187*, 103111. [\[CrossRef\]](#)
67. Breiman, L.; Friedman, J.; Stone, C.J.; Olshen, R. *Classification and Regression Trees*; Chapman and Hall/CRC: Boca Raton, FL, USA, 1984.
68. Breiman, L. Random Forests. *Mach. Learn.* **2001**, *45*, 5–32. [\[CrossRef\]](#)
69. Hinton, G.E.; Roweis, S. Stochastic Neighbor Embedding. In *Proceedings of the Advances in Neural Information Processing Systems, New Orleans, LA, USA, 28 November–9 December 2022*; Becker, S., Thrun, S., Obermayer, K., Eds.; MIT Press: Cambridge, MA, USA, 2002; Volume 15.
70. Olszewski, D.; Iwanowski, M.; Graniszewski, W. Dimensionality reduction for detection of anomalies in the IoT traffic data. *Future Gener. Comput. Syst.* **2024**, *151*, 137–151. [\[CrossRef\]](#)
71. ElSayed, M.S.; Le-Khac, N.A.; Albahar, M.A.; Jurcut, A. A novel hybrid model for intrusion detection systems in SDNs based on CNN and a new regularization technique. *J. Netw. Comput. Appl.* **2021**, *191*, 103160. [\[CrossRef\]](#)
72. Zoghi, Z.; Serpen, G. UNSW-NB15 Computer Security Dataset: Analysis through Visualization. *arXiv* **2021**, arXiv:2101.05067. [\[CrossRef\]](#)
73. Perumal, S.; Sujatha, P.K.; Krishnaa, S.; Krishnan, M. Clusters in chaos: A deep unsupervised learning paradigm for network anomaly detection. *J. Netw. Comput. Appl.* **2025**, *235*, 104083. [\[CrossRef\]](#)
74. Rai, V.; Mishra, P.K.; Joshi, S.; Kumar, R.; Dwivedi, A.; Amrita. Innovative detection of IoT cyber threats using a GBiTCN-Temformer and MKOA framework. *J. Netw. Comput. Appl.* **2025**, *240*, 104192. [\[CrossRef\]](#)
75. Asmitha, K.; P., V.; Rafidha Rehimani, K.; Raveendran, N.; Conti, M. Android malware defense through a hybrid multi-modal approach. *J. Netw. Comput. Appl.* **2025**, *233*, 104035. [\[CrossRef\]](#)

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.